

[PROPER] VOICE ASSISTANCE

VOICE RESPONSE MECHANISM

A. Key Responses and System Confirmation

This subsection presents the voice responses immediately issued by BraiVo upon user interaction with its keys and system functions.

1.1. Power Functions

1.1.1. Power On

Voice Prompt: "Calculator Ready"

1.1.2. Power Off

Voice Prompt: "Calculator Off"

1.1.3. Battery Low

Voice Prompt: "Battery low. Please recharge soon"

1.2. Editing and Navigation Keys

1.2.1. AC Key Pressed

Voice Prompt: "Cleared"

1.2.2. DEL Key Pressed

Voice Prompt: "Deleted"

1.2.3. ANS Key Pressed

Voice Prompt: "Previous Answer Inserted [Previous Answer]."

Example: "Previous Answer Inserted: 123"

1.2.4. Long Press Not Detected

Scenario: User intended to activate the Status Query Key but released the key too early.

Voice Prompt: "Long press required"

1.2.5. Multiple Keys Pressed Simultaneously

Scenario: Two or more buttons are pressed at the same time.

Voice Prompt: "Press one key at a time"

1.3. Status Query Function

1.3.1. Scenario: The Status Query key is pressed while an expression is still being entered.

Example: LCD displays $8 \times (4 + 9)$

Voice Prompt: "Current Expression: Eight times open parenthesis four plus nine close parenthesis."

1.3.2. If one or more expression errors are detected, the calculator will continue by stating:

Voice Prompt: "The working expression has an error: [Specific error message]."

Example: Working expression is $5 \times + 6$

Voice Prompt: "Current Expression: Five multiplied plus six. The working expression has an error: Two operators entered consecutively."

1.3.3. Scenario: The Status Query key is pressed after pressing the equals (=) key.

Example: LCD displays 8

Voice Prompt: "Current Result: Eight."

B. Expression Validation and Error Messages

The following error messages are evaluated only after the equals (=) key is pressed. If the entered expression satisfies any error condition, the corresponding voice prompt is issued. When the Status Query function is

used before solving, the current expression is first read aloud before announcing any detected error.

2.1. Expression Structure Errors

2.1.1. Multiple Operators

Example: $5 \times + 3$

Voice Prompt: "Invalid operation. [Two] operators entered consecutively."

[Two] can be changed depending on how many operators were erroneously placed next to each other.

2.1.2. No Operator Entered

Example: $65 =$

Voice Prompt: "No operation entered."

2.1.3. Expression Started with an Operator

Example: $+57=$

Voice Prompt: "Expression cannot begin with an operator."

2.1.4. Expression Ends with an Operator

Example: $57+$

Voice Prompt: "Incomplete Expression."

2.1.5. Equal or Operator Pressed Without Input

Example: $=$ or $+$

Voice Prompt: "No expression entered."

2.2. Decimal Validation Errors

2.2.1. Decimal Entered Twice

Example: $5.3.4$

Voice Prompt: "Only one decimal point is allowed per number."

2.2.2. Operator Immediately After a Decimal Point

Example: 5. + 5

Voice Prompt: "Incomplete decimal number."

2.3. Parentheses Errors

2.3.1. Empty Parentheses

Example: ()

Voice Prompt: "Empty parentheses are not allowed"

2.3.2. Incomplete Parentheses

Example: (8 + 7 =

Voice Prompt: "Missing closing (or opening) parenthesis."

2.4. Percentage Error

Example: %

Voice Prompt: "Percentage requires a preceding number."

2.5. ANS Key Error

Scenario: The first calculation attempts to use the ANS function before any previous result exists.

Voice Prompt: "No previous answer available."

2.6. Calculation Errors

Example: $60 \div 0$

Voice Prompt: "Current Result: Undefined, cannot divide by zero."

2.7. Result Formatting

2.7.1. Result is too large or too small

Scenario: When the calculated answer is too large or too small to display normally, BraiVo automatically reads the result in scientific notation.

Example: $1 \div 1,000,000,000 = 1 \times 10^{-9}$

Voice Prompt: “Current Result: One multiplied to ten raised to negative nine.”

2.7.2. Result has a non-terminating, repeating decimal

The calculator initially announces the result in fraction form. When the S-D (Standard-to-Decimal) option is selected, the corresponding decimal equivalent is then read aloud. If the repeating number is between the tenths and ten-thousandths, only the first four decimal places are spoken before indicating that the decimal repeats. The last number will not be rounded off.

Example 1: $1 \div 3 = 0.3333\dots$

Voice Prompt: “Current Result: Zero point three three three three, repeating.”

Example 2: $2 \div 3 = 0.6666\dots$

Voice Prompt: “Current Result: Zero point six six six six, repeating.”

Example 3: $7 \div 11 = 0.6363\dots$

Voice Prompt: “Current Result: Zero point six three six three, repeating.”

Example 4: $1 \div 101 = 0.00990099\dots$

Voice Prompt: “Current Result: Zero point zero zero nine nine, repeating.”

If the repeating number is beyond the ten-thousandths place, BraiVo will read decimals from the tenths to ten-thousandths place then state “approximately”.

Example: $1 \div 28 = 0.0357142857142857\dots$

The repeating part is 142857.

Voice Prompt: “Current Result: Zero point zero three five seven, approximately.”

2.7.3. Result has a non-terminating, non-repeating decimal

This condition cannot occur because BraiVo does not support irrational number operations such as π , square roots, or other functions that produce non-repeating, non-terminating decimal values.

2.8. Other Errors

Should any unexpected error occur that is not covered by the preceding conditions, BraiVo will issue the following message:

Voice Prompt: “Error, please rewrite your equation.”

C. Expression Continuation Behavior

If the user performs a calculation and immediately presses an arithmetic operator, the calculator starts a new expression with the selected operator instead of automatically inserting the previous result.

Example: $8 + 9 = 17$

User presses +

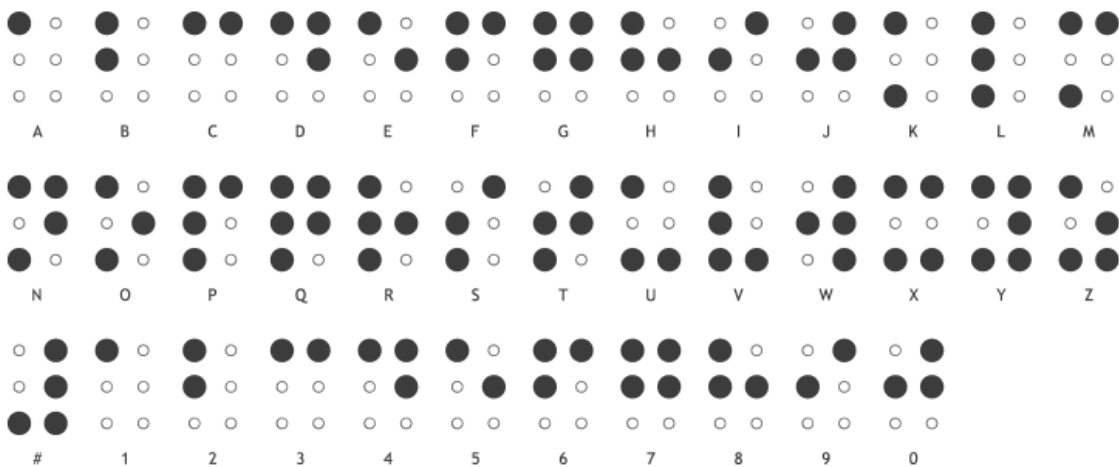
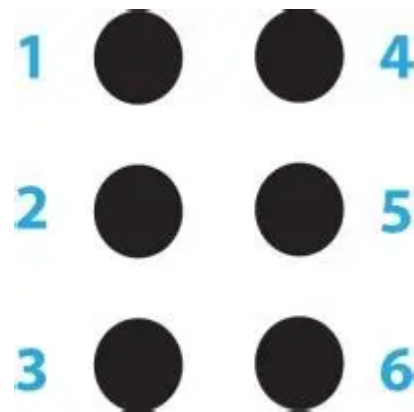
The new expressions begin with + rather than $17 +$.

To continue calculations using the previous result, the user must press the ANS key first.

Braille UEB

In Braille notation, the dot numbers for a single cell must always be listed in **ascending numerical order**

Guide for 6-dot braille system



Name	Braille Symbol	Braille Dot Numbers
------	----------------	---------------------

Numbers	Always starts with a # (3-4-5-6) then proceed with the number.	
1		Left: 3-4-5-6 Right: 1
2		Left: 3-4-5-6, Right: 1-2
3		Left: 3-4-5-6, Right: 1-4
4		Left: 3-4-5-6, Right: 1-4-5
5		Left: 3-4-5-6, Right: 1-5

6		Left: 3-4-5-6, Right: 1-2-4
7		Left: 3-4-5-6, Right: 1-2-4-5
8		Left: 3-4-5-6, Right: 1-2-5
9		Left: 3-4-5-6, Right: 2-4
0		Left: 3-4-5-6, Right: 2-4-5
Arithmetic Operations & Equals		

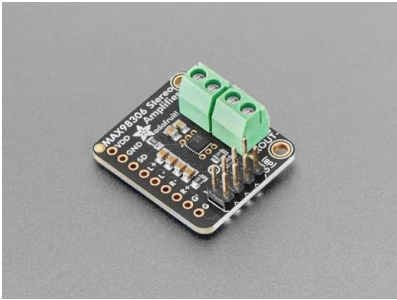
Plus	⠠⠨⠶	Left: 5 Right: 2-3-5
Minus	⠠⠨⠼	Left: 5 Right: 3-6
Multiplication cross	⠠⠨⠢	Left: 5 Right: 2-3-6
Division	⠠⠨⠴	Left: 5 Right: 3-4
Equals	⠠⠨⠸	Left: 5 Right: 2-3-5-6
Clear / Delete Keys		

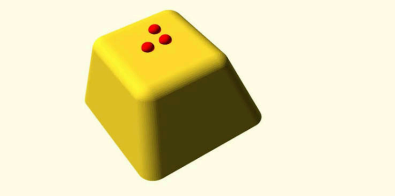
DEL (Delete)	⠠⠠⠠	Left: 1-4-5, Middle: 1-5, Right: 1-2-3
AC (All Clear)	⠠⠠	Left: 1 Right: 1-4
Decimal, Percentage & Parentheses		
Decimal point	⠠	2-5-6
Percentage	⠠⠠	Left: 4-6, Right: 3-5-6
Open Parenthesis	⠠⠠	Left: 5, Right: 1-2-6

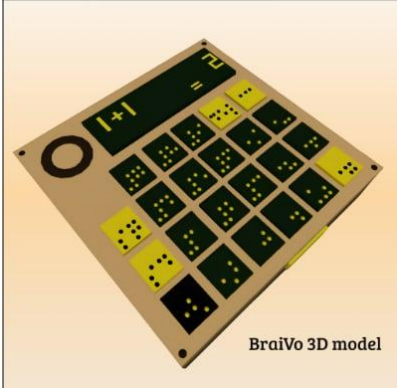
Close Parenthesis	⠨⠠⠨	Left: 5, Right: 3-4-5
Special Function Keys		
SQ (Status Query)	⠨⠠⠨⠠⠨	Left: 2-3-4, Right: 1-2-3-4-5
SD (Standard to Decimal)	⠨⠠⠨⠠⠨	Left: 2-3-4, Right: 1-4-5

Components

Complete Component List

Component	Function in Project	Picture
ESP32 Development Board	Acts as the main brain. It handles the 5x5 key matrix scanning, math logic (including PEMDAS grouping and software debounce), error condition checks, and generates the TTS audio entirely in software.	
16x2 I2C LCD Screen	Displays the high-contrast yellow text (Status, Working Equation, Results) across 2 spacious lines, requiring only 2 data pins (SDA and SCL).	
Adafruit MAX98306 Stereo Class-D Audio Amp 3.7W	Takes the analog audio signal from the ESP32's DAC pins and amplifies it to drive the speaker.	

Micro Speaker	Delivers the automated robotic voice for numbers, structural summaries, and error messages.	
23x Tactile Push Buttons	The physical switches are directly wired into a 5x5 matrix. These must have a high actuation force (250gf+) to be tough to click, preventing accidental inputs while the user reads the braille.	
Custom Keycaps	23 individual keycaps featuring raised UEB 6-dot braille cells, 3D printed in bright yellow.	
3.7V Li-Ion Battery	The internal rechargeable power source.	

TP4056 Charge Controller	Safely manages battery charging via the side port. A routed wire allows the ESP32 to monitor voltage and trigger the "Battery low" audio alert.	
5V Step-Up (Boost) Converter	Takes the 3.7V from the battery/TP4056 and boosts it to a stable 5V. This is required to power the LCD screen correctly and prevent the ESP32 from crashing when the audio plays.	
3D Printed Enclosure	The main casing, fabricated in black filament to create maximum visual contrast against the yellow braille keycaps. (Color will be changed later)	 BraiVo 3D model

Stranded Wires	Used to physically connect all components. <i>Recommendation: Use solid-core wire for routing the matrix rows/columns on the perfboard, and stranded wire for the battery and speaker connections.</i>	STRANDED 
Soldering Consumables	Standard rosin-core solder (60/40 or lead-free) and soldering flux paste to ensure secure, permanent connections between the wires and modules.	
3D Printer Filament	Standard PETG filament. Black is used to print the main enclosure for maximum visual contrast, while Bright Yellow is used to print the 23 individual keycaps. (Color will be changed later)	

PRODUCTS	QUANTITY	TOTAL
 Type C Micro USB 5V 1A 18650 TP4056 Lithium Battery Charger Module Charging Board With Protection P30.00 Model: TP4056 Micro USB (With Protection)	- 1 + 🗑️	P30.00
 PKCell 2000mAh Lithium Ion Battery - 3.7v LP803860 P400.00	- 1 + 🗑️	P400.00
 25Pcs Tactile Push Button Switch Momentary 12*12*7.3MM P150.00	- 1 + 🗑️	P150.00
 16x2 LCD Display I2C Black on Green P150.00	- 1 + 🗑️	P150.00
 Expansion Board for 30PIN 38PIN ESP32 Development Board Terminal Block Type P119.00 P149.00 Pin: 38 Pin	- 1 + 🗑️	P119.00
 30 pins and 38 pins ESP32 WiFi IoT Development Board P350.00 Types: 38 Pin	- 1 + 🗑️	P350.00

Fulfillment Instructions

Regular price	P1,229.00
Your savings	P30.00

Subtotal (6) **P1,199.00**

// items.md

2 items



SKU: 1314

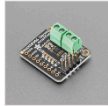
**Speaker 1.5" Diameter 4 Ohm 3W -
Adafruit 1314 Compact**

₱ 58.00

-	1	+
---	---	---

x

₱ 58.00



SKU: 987

**Adafruit MAX98306 Stereo Class-D
Audio Amp 3.7W - 987**

₱ 585.00

-	1	+
---	---	---

x

₱ 585.00

[← Continue shopping](#)

// summary.md

[+ add order note](#)

subtotal ₱ 643.00

shipping calculated at checkout

total ₱ 643.00

[Checkout →](#)

Tax + shipping calc at checkout.
Same-day ship before 4 PM weekday cutoff.

WE ACCEPT

[GCash](#) [Maya](#) [Visa](#) [MC](#) [COD](#) [BPI](#)

Secure SSL checkout

Manila stock · same-day ship 4 PM cutoff

7-day inspection window

Procedures

PROCEDURES

The development of the Audio-Assisted Braille Calculator was executed through a six-phase engineering lifecycle, progressing from initial sketches to final system benchmarking.

Phase 1: Conceptualization and Digital Design

- **Step 1: Draft Initial Layouts:** Create preliminary hand-drawn drafts to visualize the device's physical layout, focusing heavily on the ergonomics of the 23-key interface and the internal placement of the audio speaker.
- **Step 2: Develop 3D Models:** Translate these initial sketches into precise 3D models using Computer-Aided Design (CAD) software. Ensure the digital models accommodate the main high-contrast enclosure and the individual keycaps featuring raised Unified English Braille (UEB) cells.
- **Step 3: Procure Components:** Acquire the necessary hardware, including the ESP32 development board, 16x2 I2C LCD screen, Adafruit MAX98306 stereo amplifier, 3W 4-Ohm speaker, high-actuation tactile push buttons, and the TP4056 power management module.

Phase 2: Component Verification and Breadboard Testing

- **Step 1: Temporary Prototyping:** Wire all electronic modules (the ESP32, the I2C LCD, the audio amplifier, and a small sub-section of the tactile switches) together on a solderless breadboard.
- **Step 2: System Verification:** Test individual module viability to ensure all hardware components are fully functional and communicating correctly with the ESP32 microcontroller prior to permanent assembly.

Phase 3: Electronic Assembly and Integration

- **Step 1: Establish Main Power:** Mount the 3.7V Lithium-Ion battery and connect it to the **B+** and **B-** pads of the TP4056 charge controller. Connect the **OUT+** and **OUT-** pads to a 5V Step-Up Converter to ensure a stable 5V supply for the logic components.
- **Step 2: Wire the Voltage Monitor:** Route a wire from the battery's positive terminal through a two-resistor voltage divider circuit to the ESP32's **GPIO 34**. This establishes the monitoring line to trigger the "Battery low" audio alert.
- **Step 3: Construct the Keypad Matrix:** Solder the 23 tactile push buttons into a custom 5x5 matrix PCB using direct wiring.
 - Wire the 5 horizontal row lines to the ESP32's **GPIO 13, 14, 27, 32, and 33**.
 - Wire the 5 vertical column lines to the ESP32's **GPIO 15, 4, 16, 17, and 5**.
- **Step 4: Integrate the LCD:** Connect the 16x2 I2C LCD screen. Power it using the 5V line and wire the **SDA** data pin to the ESP32's **GPIO 21** and the **SCL** clock pin to **GPIO 22**.
- **Step 5: Integrate the Audio System:** Wire the MAX98306 amplifier's **INL+** and **INR+** analog input pins directly to the ESP32's internal DAC pin at **GPIO 25**. Connect the negative input pins to Ground, and wire the 3W speaker to the amplifier's output terminals.

Phase 4: 3D Printing and Enclosure Assembly

- **Step 1: Fabricate the Housing:** Export the digital CAD models and 3D print the main casing using black filament for maximum visual contrast. Print the 23 braille keycaps using bright yellow filament.
- **Step 2: Mount Internal Hardware:** Carefully secure the soldered electronic assemblies and the battery into their designated holders inside the main casing.
- **Step 3: Finalize External Interface:** Mount the LCD screen and the speaker to their respective cutouts on the top panel. Snap the 3D-printed braille keycaps onto the tactile switch stems to complete the physical hardware.

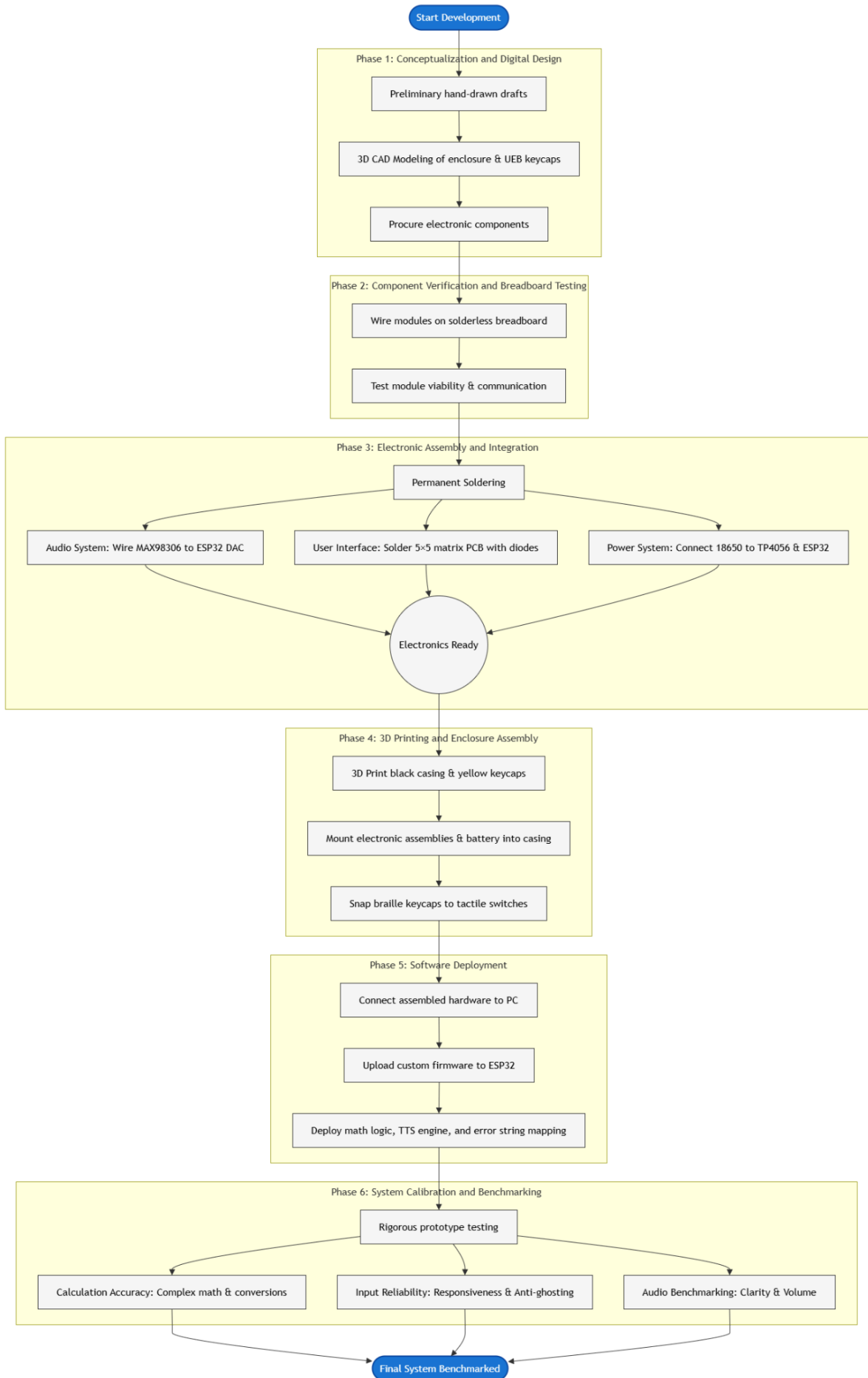
Phase 5: Software Deployment

- **Step 1: Initialize Microcontroller:** Connect the fully assembled ESP32 to a computer via a USB data cable.
- **Step 2: Deploy Core Logic:** Upload the custom firmware containing the mathematical calculation logic, PEMDAS grouping algorithms, and matrix scanning logic. Ensure a 50-millisecond software debounce sequence is active to prevent false double-clicks on the direct-wired keypad.
- **Step 3: Configure the TTS Engine:** Deploy the Text-to-Speech (TTS) engine and structural error detection logic. Map specific structural errors to their exact audio strings (e.g., mapping a decimal entered twice to output *"Only one decimal point is allowed per number"*).

Phase 6: System Calibration and Benchmarking

- **Step 1: Audio Benchmarking:** Test the TTS output for clarity and volume, verifying that the MAX98306 amplifier outputs a robotic voice that is easily understandable without distortion.
- **Step 2: Input Reliability Testing:** Benchmark the 5x5 keypad for tactile responsiveness and evaluate the software debounce accuracy to ensure the system successfully flags simultaneous key presses with the warning *"Press one key at a time"*.
- **Step 3: Calculation Accuracy Validation:** Test the device against complex mathematical expressions to benchmark the speed and accuracy of the ESP32's processing, ensuring non-terminating decimals correctly limit their spoken output to the first 4 decimal places.

System Architecture

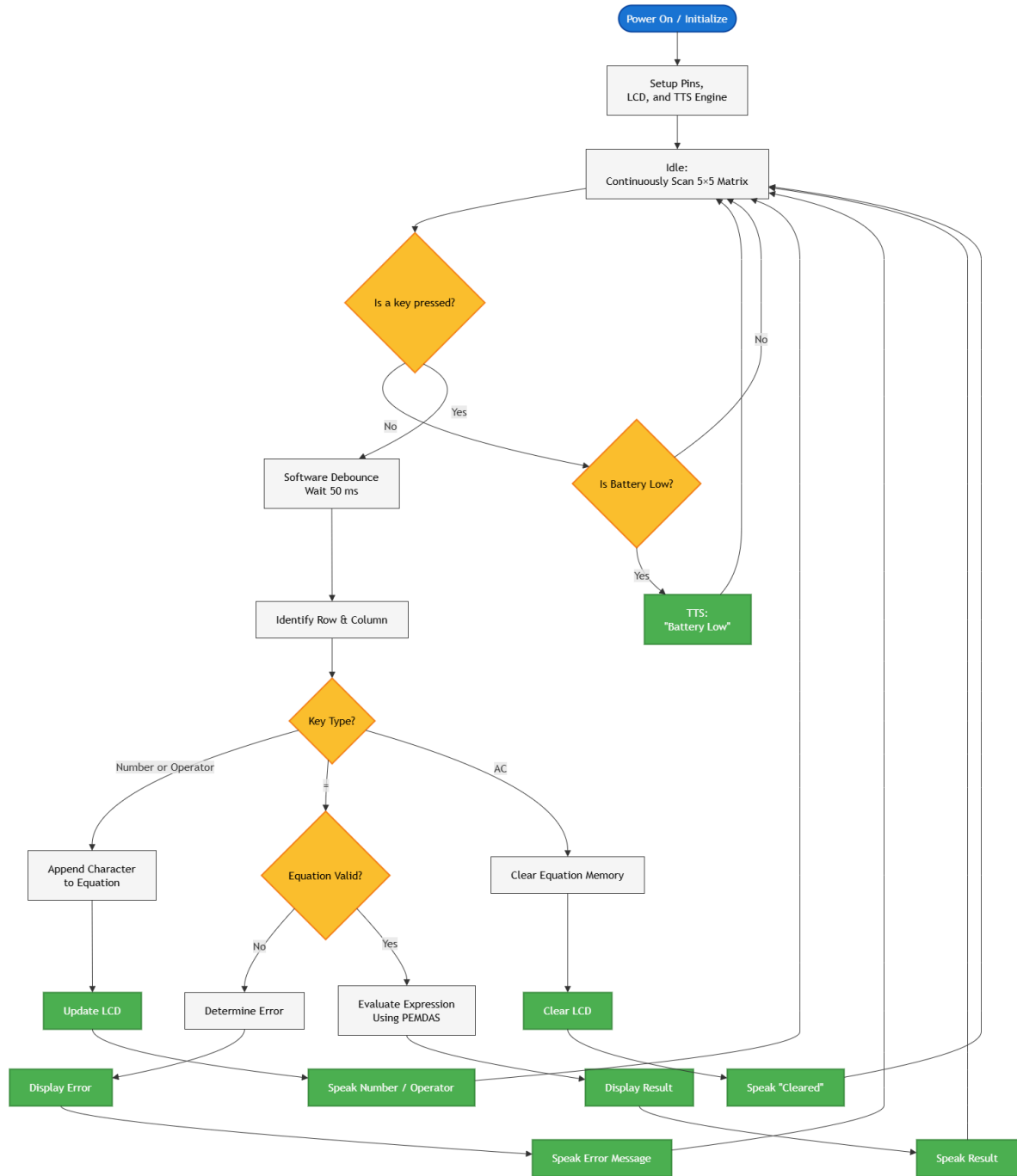


The development process of the proposed system follows a structured six-phase methodology to ensure that both the hardware and software components are systematically designed, integrated, and evaluated. The process begins with Conceptualization and Digital Design, where preliminary hand-drawn sketches are transformed into detailed 3D CAD models of the enclosure and Unified English Braille (UEB) keycaps while the required electronic components are identified and procured. This is followed by Component Verification and Breadboard Testing, during which all electronic modules are assembled on a solderless breadboard to verify functionality, compatibility, and communication before permanent assembly.

Once the components have been validated, the project proceeds to Electronic Assembly and Integration, where permanent soldering is performed to build the power system, user interface, and audio subsystem. These integrated electronic modules are then installed during the 3D Printing and Enclosure Assembly phase, where the custom-designed enclosure and Braille keycaps are fabricated and assembled into a complete hardware prototype. After the physical assembly is completed, Software Deployment is carried out by uploading the custom firmware to the ESP32 microcontroller and implementing the calculator logic, text-to-speech functionality, and error-handling mechanisms. Finally, the prototype undergoes System Calibration and Benchmarking, where extensive testing is conducted to evaluate audio clarity, input responsiveness, anti-ghosting performance, and computational accuracy. The successful completion of these phases results in a fully assembled, programmed, and benchmarked prototype that is ready for validation and usability evaluation.

Schematic Diagram

Flow Chart



This flowchart outlines the primary programming logic and sequential workflow of the microcontroller used in the Audio-Assisted Braille Calculator.

The sequence begins with system initialization, setting up the hardware pins, the LCD display, and the Text-to-Speech (TTS) engine. The system then enters its

primary "Idle" state, where it continuously scans the 5x5 keypad matrix for user input. During this idle loop, it simultaneously checks if the battery is low, triggering a "Battery Low" audio warning if the voltage drops below a certain threshold.

When a key press is detected, the logic immediately applies a 50-millisecond software debounce to prevent false double-clicks before identifying the specific row and column pressed. The system's response then branches into three possible paths based on the type of key pressed:

1. **Numbers and Operators:** The character is appended to the working equation, updated on the LCD screen, and spoken aloud by the TTS engine before returning to the idle loop.
2. **The "Clear" (AC) Key:** The system wipes the equation from memory, clears the LCD screen, and provides an audio confirmation stating "Cleared."
3. **The "Equals" (=) Key:** The system first evaluates whether the inputted equation is structurally valid. If valid, it calculates the expression using PEMDAS rules, displays the final answer, and speaks the result. If the equation contains structural errors (like consecutive operators), it intercepts the calculation, determines the specific error, and outputs the corresponding visual and audio error messages.